

17-3. Testing `python_repos.py`: In `python_repos.py`, we printed the value of `status_code` to make sure the API call was successful. Write a program called `test_python_repos.py` that uses `pytest` to assert that the value of `status_code` is 200. Figure out some other assertions you can make: for example, that the number of items returned is expected and that the total number of repositories is greater than a certain amount.

17-4. Further Exploration: Visit the documentation for Plotly and either the GitHub API or the Hacker News API. Use some of the information you find there to either customize the style of the plots we've already made or pull some different information and create your own visualizations. If you're curious about exploring other APIs, take a look at the APIs mentioned in the GitHub repository at <https://github.com/public-apis>.

Summary

In this chapter, you learned how to use APIs to write self-contained programs that automatically gather the data they need and use that data to create a visualization. You used the GitHub API to explore the most-starred Python projects on GitHub, and you also looked briefly at the Hacker News API. You learned how to use the `Requests` package to automatically issue an API call and how to process the results of that call. We also introduced some Plotly settings that further customize the appearance of the charts you generate.

In the next chapter, you'll use Django to build a web application as your final project.